LAPORAN PRAKTIKUM MACHINE LEARNING
K_NEAREST_NEIGHBORS



Oleh:

Nama : Dyelvio Wijaya

NIM : 211344012

Tanggal Praktikum : 27 Oktober 2023

Tanggal Pengumpulan Laporan : 31 Oktober 2023

PRODI D4 TEKNIK TELEKOMUNIKASI - TEKNIK ELEKTRO
POLITEKNIK NEGERI BANDUNG
2023/2024

A. Program K-Nearest Neighbors

Import Library

```
[1]  import numpy as np
     import matplotlib.pyplot as plt
     import pandas as pd
     import numpy as np
     from sklearn import preprocessing
     %matplotlib inline
```

Dowload Data

```
!wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-ML0101EN-SkillsNetwork/labs/Module%203/data/teleCust1000t.csv"

--2023-10-29 03:35:24--  https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-ML0101EN-SkillsNetwork/labs/Module%203/data/teleCust1000t.csv
Resolving cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud)... 169.45.118.108
Connecting to cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud)|169.45.118.108|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 36047 (35K) [text/csv]
Saving to: 'teleCust1000t.csv'

teleCust1000t.csv   100%[===================>]  35.20K  --.-KB/s    in 0.03s

2023-10-29 03:35:25 (1.02 MB/s) - 'teleCust1000t.csv' saved [36047/36047]
```

Load Data dari file CSV

```
df = pd.read_csv('teleCust1000t.csv')
df.head()
```

	region	tenure	age	marital	address	income	ed	employ	retire	gender	reside	custcat
0	2	13	44	1	9	64.0	4	5	0.0	0	2	1
1	3	11	33	1	7	136.0	5	5	0.0	0	6	4
2	3	68	52	1	24	116.0	1	29	0.0	1	2	3
3	2	33	33	0	12	33.0	2	0	0.0	1	1	1
4	2	23	30	1	9	30.0	1	2	0.0	0	4	3

Data Visualization and Analysis

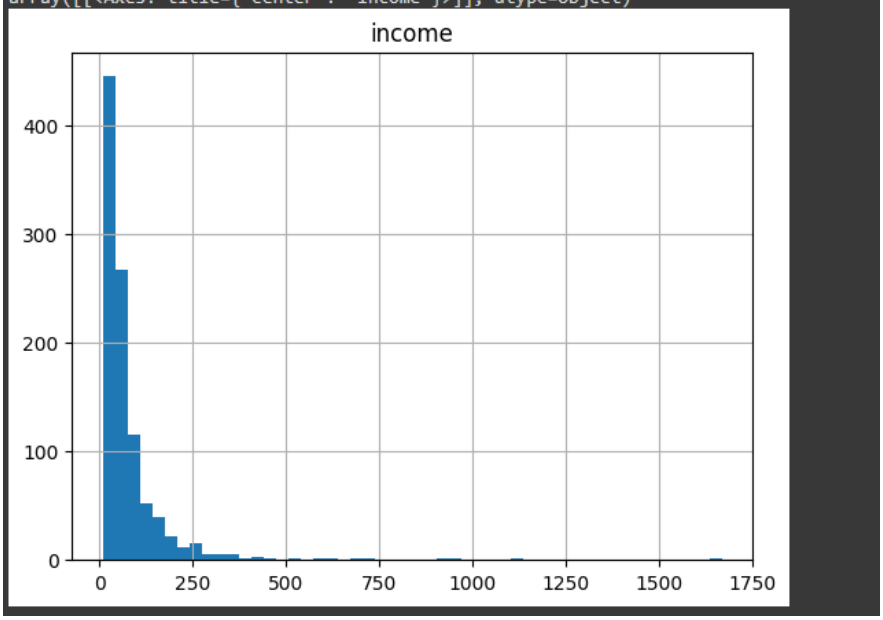
```
#"untuk menghitung jumlah kemunculan setiap nilai unik dalam kolom 'custcat' dari DataFrame df"
df['custcat'].value_counts()
```

```
3    281
1    266
4    236
2    217
Name: custcat, dtype: int64
```

Menampilkan Histogram

```
#"Histogram akan membagi rentang pendapatan ke dalam 50 interval untuk data income"
df.hist(column='income', bins=50)
```

```
array([[<Axes: title={'center': 'income'}>]], dtype=object)
```



Feature Set

```
df.columns#untuk memberi tahu nama nama kolom pada dataframe atau df
```

```
Index(['region', 'tenure', 'age', 'marital', 'address', 'income', 'ed',
       'employ', 'retire', 'gender', 'reside', 'custcat'],
      dtype='object')
```

```
X = df[['region', 'tenure','age', 'marital', 'address', 'income', 'ed', 'employ','retire', 'gender', 'reside']] .values  #.astype(float)
X[0:5] #"Menampilkan nilai X dari indeks 0 - 5 dari setiap data"
```

```
array([[  2.,  13.,  44.,   1.,   9.,  64.,   4.,   5.,   0.,   0.,   2.],
       [  3.,  11.,  33.,   1.,   7., 136.,   5.,   5.,   0.,   0.,   6.],
       [  3.,  68.,  52.,   1.,  24., 116.,   1.,  29.,   0.,   1.,   2.],
       [  2.,  33.,  33.,   0.,  12.,  33.,   2.,   0.,   0.,   1.,   1.],
       [  2.,  23.,  30.,   1.,   9.,  30.,   1.,   2.,   0.,   0.,   4.]])
```

```
y = df['custcat'].values
y[0:5] #"Menampilkan nilai y dari data custcat dari indeks 0 - 5"
```

```
array([1, 4, 3, 1, 3])
```

Membagi Trainset dan Testset

```
#Program untuk membagi bagi train data dan juga data set dari dataframe
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=4)
print ('Train set:', X_train.shape,  y_train.shape)
print ('Test set:', X_test.shape,  y_test.shape)
```

```
Train set: (800, 11) (800,)
Test set: (200, 11) (200,)
```

Normalisasi Data

```
#"preprocessing.StandardScaler() membuat objek yang akan digunakan untuk melakukan penskalaan standar"
#"fit() untuk menghitung rata-rata dan deviasi standar dari setiap kolom (fitur) dalam data 'X_train'"
#"transform() untuk menerapkan penskalaan standar pada data 'X_train' berdasarkan rata-rata
# dan deviasi standar(dalam tipe data float)"
X_train_norm = preprocessing.StandardScaler().fit(X_train).transform(X_train.astype(float))
X_train_norm[0:5]
```

```
array([[-1.28618818e+00, -1.53085556e+00, -8.49354628e-01,
         1.00752834e+00, -7.46393214e-01, -4.83608776e-01,
         1.12305195e+00, -7.02283455e-01, -2.23313158e-01,
        -1.01005050e+00,  1.83304333e+00],
       [-4.79805457e-02,  2.51253999e-01,  3.44957012e-01,
         1.00752834e+00,  2.45326718e-01, -2.71543263e-01,
        -5.40728715e-01, -1.25385370e-04, -2.23313158e-01,
        -1.01005050e+00,  1.83304333e+00],
       [-1.28618818e+00, -7.80493639e-01, -6.90113076e-01,
         1.00752834e+00, -5.21892614e-02, -4.92444839e-01,
        -1.37261905e+00, -8.02591750e-01, -2.23313158e-01,
        -1.01005050e+00,  1.83304333e+00],
       [-4.79805457e-02, -1.53085556e+00, -3.71629972e-01,
        -9.92527915e-01,  1.46154725e-01, -4.74772713e-01,
        -1.37261905e+00, -8.02591750e-01, -2.23313158e-01,
         9.90049504e-01, -9.33814526e-01],
       [ 1.19022709e+00, -2.17722200e-01, -6.10492300e-01,
        -9.92527915e-01, -2.50533248e-01,  2.40948394e-01,
         1.12305195e+00, -4.01358568e-01, -2.23313158e-01,
         9.90049504e-01,  1.14132887e+00]])
```

Classification

1. Import Library

```
from sklearn.neighbors import KNeighborsClassifier
```

2. Training

```
k = 4
#Train Model and Predict
#KNeighborsClassifier(n_neighbors = k) argumen yang menentukan jumlah tetangga terdekat
#yang akan digunakan dalam pemodelan
#.fit() digunakan untuk melatih model
neigh = KNeighborsClassifier(n_neighbors = k).fit(X_train_norm,y_train)
neigh
```

```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=4)
```

```
X_test_norm = preprocessing.StandardScaler().fit(X_test).transform(X_test.astype(float))
X_test_norm[0:5]
```

```
array([[-1.13100942, -0.93533648, -0.93817795,  1.02020406, -0.36233308,
         0.17102716, -0.58336825,  0.00666863, -0.21707238, -1.13967126,
         1.97590703],
       [-1.13100942, -1.44823286, -0.13972863, -0.98019606,  0.03474427,
        -0.50017376, -0.58336825, -0.56492821, -0.21707238, -1.13967126,
        -0.90337462],
       [-1.13100942, -0.51569399,  1.53701494,  1.02020406, -0.56087175,
         0.45499678,  0.9619781 , -0.75546048, -0.21707238,  0.87744601,
         1.25608662],
       [-1.13100942, -0.74882871, -0.85833302,  1.02020406, -1.15648777,
        -0.70669712,  0.9619781 , -0.85072662, -0.21707238, -1.13967126,
         0.53626621],
       [-1.13100942,  0.51009876, -1.01802288,  1.02020406, -0.75941043,
         0.17102716,  1.73465128, -0.75546048, -0.21707238, -1.13967126,
         1.25608662]])
```

3. Predicting

```
#"melakukan prediksi menggunakan model K-Nearest Neighbors (K-NN) yang telah dibuat sebelumnya"
yhat = neigh.predict(X_test_norm)
yhat[0:5]
```

```
array([3, 1, 3, 2, 4])
```

4. Accuracy Evaluation

```
from sklearn import metrics
#untuk menghitung akurasi dari model K-Nearest Neighbors (K-NN)
#pada data pelatihan (X_train_norm) dan label kelas yang sesuai (y_train)
print("Train set Accuracy: ", metrics.accuracy_score(y_train, neigh.predict(X_train_norm)))
#untuk menghitung akurasi dari model K-Nearest Neighbors (K-NN)
#pada data uji (y_test) berdasarkan hasil prediksi model (yhat)
print("Test set Accuracy: ", metrics.accuracy_score(y_test, yhat))
```

```
Train set Accuracy:  0.5475
Test set Accuracy:  0.33
```

5. Practice

```
# write your code here
from sklearn import metrics
k = 6 #Mengunakan k = 6
neigh = KNeighborsClassifier(n_neighbors = k).fit(X_train_norm,y_train)
yhat = neigh.predict(X_test_norm)
print("Train set Accuracy: ", metrics.accuracy_score(y_train, neigh.predict(X_train_norm)))
print("Test set Accuracy: ", metrics.accuracy_score(y_test, yhat))
```

```
Train set Accuracy:  0.5125
Test set Accuracy:  0.335
```

6. Nilai K 1 – 9

```
#menguji model K-NN dengan jumlah tetangga terdekat dari 1 hingga 9
Ks = 10
mean_acc = np.zeros((Ks-1))
std_acc = np.zeros((Ks-1))

for n in range(1,Ks):

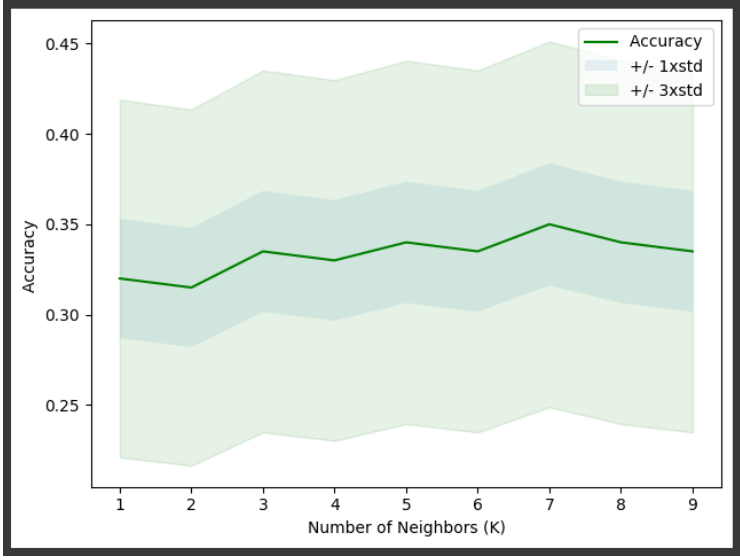
    #Train Model and Predict
    neigh = KNeighborsClassifier(n_neighbors = n).fit(X_train_norm,y_train)
    yhat=neigh.predict(X_test_norm)
    mean_acc[n-1] = metrics.accuracy_score(y_test, yhat)


    std_acc[n-1]=np.std(yhat==y_test)/np.sqrt(yhat.shape[0])
#Menampilkan nilai acuracy dari k = 1 - 9
mean_acc
```

```
array([0.32 , 0.315, 0.335, 0.33 , 0.34 , 0.335, 0.35 , 0.34 , 0.335])
```

7. Plot Model Accuracy dengan K yang berbeda

```
#plt.plot(range(1, Ks), mean_acc, 'g'): Ini adalah perintah untuk membuat plot garis berdasarkan data
plt.plot(range(1,Ks),mean_acc,'g')
#untuk mengisi area antara dua garis yang sejajar dengan garis akurasi rata-rata (mean_acc) dengan lebar satu deviasi standar (std_acc)
plt.fill_between(range(1,Ks),mean_acc - 1 * std_acc,mean_acc + 1 * std_acc, alpha=0.10)
#untuk mengisi area antara dua garis yang sejajar dengan garis akurasi rata-rata (mean_acc) dengan lebar tiga deviasi standar (std_acc)
plt.fill_between(range(1,Ks),mean_acc - 3 * std_acc,mean_acc + 3 * std_acc, alpha=0.10,color="green")
#menambahkan legenda ke grafik yang menjelaskan elemen-elemen yang ditampilkan dalam grafik
plt.legend(('Accuracy ', '+/- 1xstd','+/- 3xstd'))
plt.ylabel('Accuracy ')
plt.xlabel('Number of Neighbors (K)')
#mengoptimalkan tata letak (layout) plot yang akan ditampilkan
plt.tight_layout()
#Menampilkan Plot
plt.show()
```



8. Nilai K terbaik

```
#mean_acc.max(): Ini adalah ekspresi yang digunakan untuk mengambil nilai maksimum dari array mean_acc
#mean_acc.argmax() + 1: Ini adalah ekspresi yang mengambil indeks dari nilai maksimum dalam array mean_acc(nilai k nya)
print( "The best accuracy was with", mean_acc.max(), "with k=", mean_acc.argmax()+1)
```

```
The best accuracy was with 0.35 with k= 7
```

B. Analisa

K-Nearest Neighbors (K-NN) adalah salah satu algoritma dalam machine learning yang digunakan untuk tugas-tugas klasifikasi dan regresi. Algoritma ini termasuk dalam kategori algoritma berbasis instans (instance-based) atau lazy learning, yang berarti algoritma ini "mengingat" train data dan membuat prediksi berdasarkan kedekatan atau kemiripan antara test data dengan train data. Dalam membuat model K-Nearest Neighbors diperlukan library numpy, matplotlib.pyplot, pandas, numpy, sklearn untuk preprocessing dan juga KneighborsClassifier. Dalam menganalisis sebuah data kita dapat menggunakan dua buah cara yaitu melihat dengan histogram atau menghitung jumlah nilai yang unik. Dapat menggunakan fungsi hist() ataupun fungsi value_counts(). Sama seperti praktik sebelumnya dalam proses pembuatan model kita perlu membagi terlebih dahulu data menjadi dua bagian yaitu untuk train data dan test data. Train data perlu memiliki data set yang lebih banyak atau lebih baik diatas 60% dataset. Namun untuk Testset hanya diperlukan sekitar 10 – 40% dikarenakan perlu juga adanya pengetesan untuk menilai apakah model yang kita buat sudah baik atau tidaknya. Layaknya manusia yang perlu banyak belajar agar saat nantinya saat test akan menghasilkan nilai yang bagus, begitu juga dengan machine learning pun sama seperti manusia sedang mempersiapkan test.

Kemudian dalam KNN perlu adanya normalisasi ini dikarenakan sensitivitas terhadap skala, K-NN mengukur kedekatan antara data dengan menghitung jarak antara titik data. Jika fitur-fitur memiliki skala yang berbeda-beda (misalnya, satu fitur dalam rentang 0-1 dan yang lain dalam rentang 0-1000), algoritma K-NN cenderung memberikan bobot yang tidak seimbang terhadap setiap fitur. Lalu karena perbandingan jarak, dalam perhitungan jarak (misalnya, jarak Euclidean), fitur-fitur dengan skala besar akan memberikan kontribusi yang lebih besar dalam mengukur jarak daripada fitur dengan skala kecil. Hal ini dapat mengakibatkan perbedaan skala antar fitur menyebabkan perbedaan besar dalam pengukuran jarak. Kemudian karena efek outlier yaitu fitur-fitur dengan skala besar juga dapat meningkatkan dampak outlier. Outlier pada satu fitur dengan skala besar dapat sangat mempengaruhi jarak keseluruhan. Sehingga Normalisasi ini akan membuat hasil dari K-NN lebih konsisten dan tidak tergantung pada perubahan skala fitur. Ini membuat model lebih dapat diandalkan dan mudah untuk diinterpretasikan.

Kemudian dalam praktiknya kali ini setiap k memiliki nilai akurasi yang berbeda beda, untuk nilai akurasi yang baik adalah nilai yang akurasinya besar, dan dalam percobaan didapatkan akurasi tertinggi sebesar 0.35 yaitu dengan k = 7. Karena dalam proses pengecekan baik atu tidaknya akurasi dari model yang kita buat kali ini mirip dengan jaccard_score function sehingga dapat diketahui jika nilai akurasi yang dihasilkan mendekati maka akan semakin bagus, karena Nilai 0 berarti tidak ada kesamaan antara dua himpunan (set) yang diukur dan nilai 1 berarti kedua himpunan sepenuhnya identik atau sama persis. Sehingga dalam percobaan kali ini nilai akurasi terendah didapatkan dengan nilai k = 2. Kemudian dalam melihat sebuah hasil akurasi dari banyaknya k dapat menggunakan langsung penampilan nilai array atau dengan memplotting nilai array tersebut kedalam sebuah grafik.